K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

DATA STRUCTURES

ASSIGNMENT – 4

TITLE : Data Management Mini Toolkit (DMMT)

SUMBITTED TO :

MRS. NEETU CHAUHAN

SUMBITTED BY :

NAME : SANIA KAMBOJ

PROGRAM :

BTECH CSE (AI & ML)

ROLL NO : 2501730319

SECTION : A

SEMESTER : 2nd

BATCH : 2025-2029

1. Introduction

Data structures are used to organize and manage data efficiently. Non-linear and advanced data structures are widely used in software systems, databases, networking, and search engines.

In this assignment, I developed a **Data Management Mini Toolkit (DMMT)** in Python. It includes three important modules:

- Binary Search Tree (BST)
- Graph Traversal using BFS and DFS
- Hash Table with Separate Chaining

These modules demonstrate searching, traversal, and fast data retrieval techniques.

---

2. Objectives

The objectives of this assignment are:

- To implement non-linear data structures practically.
- To understand tree operations like insertion, search, deletion, and traversal.
- To perform graph traversal using BFS and DFS.
- To implement hashing and collision handling.
- To connect data structures with real-world applications.

---

3. Modules Implemented

3.1 Binary Search Tree (BST)

A Binary Search Tree stores data in sorted form.

Operations Performed:

- Insert nodes
- Search nodes

- Delete nodes
- Inorder Traversal

Test Data Used:

Inserted nodes:

```
50, 30, 70, 20, 40, 60, 80
```

Search Results:

- Search 20 → Found
- Search 90 → Not Found

Deletion Cases:

- Delete leaf node → 20
- Delete one child node → 60
- Delete two children node → 30

Inorder Traversal:

```
20 30 40 50 60 70 80
```

Why Inorder Gives Sorted Output?

Inorder traversal visits:

- Left subtree
- Root
- Right subtree

Since BST stores smaller values on left and larger values on right, inorder traversal prints sorted data.

Complexity:

- Average Case: O(log n)
- Worst Case: O(n) if tree becomes skewed.

3.2 Graph Module (BFS and DFS)

A graph represents connections between nodes.

Graph Used:

Nodes: A, B, C, D, E, F

Edges:

- A → B (2)
- A → C (4)
- B → D (7)
- B → E (3)
- C → E (1)
- C → F (8)
- D → F (5)
- E → D (2)
- E → F (6)

Adjacency List:

A → B, C
B → D, E
C → E, F
D → F
E → D, F
F → None

BFS Traversal from A:

```
A  B  C  D  E  F
```

DFS Traversal from A:

```
A  B  D  F  E  C
```

Difference Between BFS and DFS:

BFS	DFS
Visits level by level	Goes deep first
Uses Queue	Uses Stack/Recursion

Real World Uses:

- BFS → Shortest path in unweighted graph
- DFS → Cycle detection, maze solving

3.3 Hash Table with Separate Chaining

Hash tables are used for fast key-value storage.

Hash Function Used:

```
index = key % table_size
```

Table Size:

5

Keys Inserted:

10, 15, 20, 7, 12

Collision Example:

- 10 % 5 = 0
- 15 % 5 = 0
- 20 % 5 = 0

All stored in same bucket.

Retrieval:

- Get(10) → Value Found
- Get(15) → Value Found

- Get(7) → Value Found

Deletion:

Deleted key 15 from collided bucket.

What is Collision?

When two or more keys get same index, collision occurs.

Why Separate Chaining Helps?

Each bucket stores multiple values in list form, so collisions are handled easily.

Complexity:

- Average Case: O(1)
- Worst Case: O(n)

---

4. Observations

- BST provides efficient ordered storage.
- BFS and DFS help explore networks.
- Hash tables give very fast searching and retrieval.
- Collision handling is necessary for efficient hashing.

---

5. Conclusion

This assignment helped me understand advanced data structures practically.

- BST is useful in databases and indexing.
- Graphs are useful in routing, maps, and social networks.
- Hash tables are used in caching and dictionaries.

These data structures are important for building efficient real-world applications.

6. References

- Python Documentation
- Data Structures Notes
- Online Learning Resources on Trees, Graphs, and Hashing